1.1 What is PROLOG?

PROLOG stands for PROgramming in LOGic. It is a logic programming language. It is mainly used in Artificial Intelligence (AI) and computational linguistics.

Unlike normal programming languages like C, C++, or Java, PROLOG is declarative, which means it focuses on what the problem is, instead of how to solve it step by step.

Key features of PROLOG

- It uses facts, rules, and queries to represent knowledge.
- It can automatically search for answers using backtracking.
- It can perform pattern matching to find solutions.
- It supports recursion, which means a rule can call itself.
- It is based on first-order predicate logic, which is used to represent knowledge clearly.

PROLOG is very useful for AI because it can reason logically and solve problems automatically.

1.2 Basics of PROLOG (Hand Notes)

1. Facts

Facts are statements that are always true.
Example:

parent(john, mary).

This means John is the parent of Mary.

2. Rules

Rules are conditional statements. They are used to find new information from known facts.
Example:

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

This means X is a grandparent of Z if X is a parent of Y and Y is a parent of Z.

3. Queries

Queries are questions asked to PROLOG.
Example:

?- parent(john, mary).

PROLOG will answer true or false.

4. Variables

- Variables start with capital letters.
  Example: X, Y, Result
- Variables can hold unknown values that PROLOG tries to find.

5. Backtracking

PROLOG can automatically try all possibilities to find a solution. If a path does not work, it goes back and tries another path. This is called backtracking.

1.3 Applications of PROLOG in Artificial Intelligence

PROLOG is widely used in AI for solving many problems. Some examples are:

1. Expert systems – systems that make decisions like a human expert.
2. Knowledge representation – storing information in a way that a computer can understand.
3. Natural language processing – understanding and generating human language.
4. Automated theorem proving – proving logical statements automatically.
5. Planning and scheduling – finding the best way to do tasks.
6. Game playing – solving logic-based games.
7. State-space search problems – finding the solution by exploring all possible states.

1.4 What is the Water-Jug Problem in AI?

The Water-Jug Problem is a classic problem in AI. It is a state-space search problem, which means it is solved by exploring all possible states.

Problem Description

- We have two jugs with fixed capacities.
- There is no measuring scale.
- We can do three types of operations:
    1. Fill a jug completely.
    2. Empty a jug completely.
    3. Pour water from one jug to another until either the first jug is empty or the second jug is full.
- The goal is to measure a specific quantity of water.

Example

- Jug A can hold 4 liters
- Jug B can hold 3 liters
- Goal: Measure 2 liters in any jug

This problem can be solved using logical reasoning and state exploration, which is why PROLOG is very useful for it.

1.5 State Representation

We can represent the state of water in both jugs as:

state(X, Y)

Where:

- X = water in Jug A
- Y = water in Jug B
- Initial state:

state(0,0)

(Both jugs are empty)

- Goal state:

state(2,_)

(One of the jugs has 2 liters of water)

1.6 PROLOG Program to Solve Water-Jug Problem

We use PROLOG to automatically find the steps to reach the goal.

Problem Setup

- Jug A = 4 liters
- Jug B = 3 liters
- Goal = 2 liters

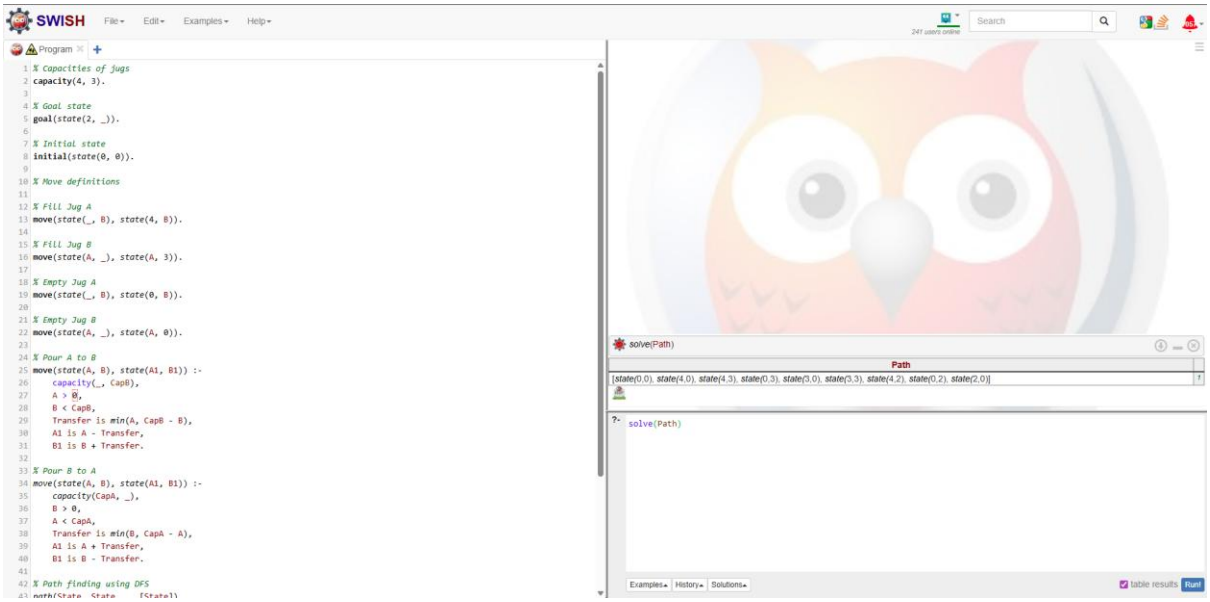
PROLOG Code

```
% Capacities of jugs
capacity(4, 3).

% Goal state
goal(state(2, _)).

% Initial state
initial(state(0, 0)).

% Move definitions
```

```
% Fill Jug A
move(state(_, B), state(4, B)).

% Fill Jug B
move(state(A, _), state(A, 3)).

% Empty Jug A
move(state(_, B), state(0, B)).

% Empty Jug B
move(state(A, _), state(A, 0)).

% Pour A to B
move(state(A, B), state(A1, B1)) :-
    capacity(_, CapB),
    A > 0,
    B < CapB,
    Transfer is min(A, CapB - B),
    A1 is A - Transfer,
    B1 is B + Transfer.

% Pour B to A
move(state(A, B), state(A1, B1)) :-
    capacity(CapA, _),
    B > 0,
    A < CapA,
    Transfer is min(B, CapA - A),
    A1 is A + Transfer,
    B1 is B - Transfer.

% Path finding using DFS
```

```
path(State, State, _, [State]).
path(State, Goal, Visited, [State|Path]) :-
    move(State, Next),
    \+ member(Next, Visited),
    path(Next, Goal, [Next|Visited], Path).

% Solve the problem
solve(Path) :-
    initial(Start),
    goal(Goal),
    path(Start, Goal, [Start], Path).
```

1.7 Output



Query

?- start(Path)

Output

state(4,0)

state(1,3)

state(1,0)

state(0,1)

state(4,1)

state(2,3)

Goal Reached: state(2,3)

true.

Explanation of Output:

- First, Jug A is filled to 4 liters.
- Then, water is poured to Jug B until Jug B is full.
- Steps continue using rules until 2 liters are measured in Jug A or Jug B.
- PROLOG finds a solution automatically using backtracking.

1.8 Conclusion

- PROLOG is very good for AI problems because it can think logically and explore possibilities automatically.
- The Water-Jug problem is a simple example of state-space search.
- PROLOG uses:
    - Facts to know the rules
    - Rules to define moves
    - Backtracking to try all options
- It efficiently finds the solution without the programmer needing to check every step manually.

Key Point: PROLOG is best for problems that require logic, reasoning, and exploring many possibilities.

2023001492

P KARTIKEYA RAJ